

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

Tres ejercicios de programación

1. Define una función `intercalar :: [a] -> [a] -> [a]` que combine dos listas alternando sus elementos uno a uno. Si una lista se agota antes que la otra, el resto de la más larga se añade al final. Si tu solución tiene tres ecuaciones, intenta conseguir una solo con dos ecuaciones (y sin `if`, `case`, ni guardas).

- `intercalar [1, 3, 5] [20, 10, 30] ~> [1, 20, 3, 10, 5, 30]`
- `intercalar [1, 2] [10, 20, 30, 40] ~> [1, 10, 2, 20, 30, 40]`
- `take 6 (intercalar [1..] [10, 20..]) ~> [1, 10, 2, 20, 3, 30]`

2. Esta función devuelve la posición de la última aparición de un valor en una lista o `-1` si no aparece: `ulti 2 [1, 2, 1, 2, 4] ~> 3`. Esta definición no es muy buena: piensa por qué y hazlo mejor.

```
ulti _ [] = -1
ulti x xs = if last xs == x
             then length xs - 1
             else ulti x (init xs)
```

3. Escribe una función que, dada una lista, devuelva un booleano que indique si estaba ordenada de menor a mayor. Haz dos versiones: una con recursión explícita y otra sin ella. Piensa en esta forma de hacerlo: toma dos copias de la lista, escribe una sobre la otra, desplaza la de abajo una posición hacia la izquierda y compara cada número con el que tiene debajo. Por ejemplo, con `[1, 2, 3, 2, 4]`:

```
  1  2  3  2  4
1  2  3  2  4
```